Assignment 1 matrix add sub mul transpose

```
#include <stdio.h>
void addition(int m, int n, int A[m][n],int B[m][n])
{
int C[m][n];
int i,j;
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
C[i][j]=A[i][j]+B[i][j];
}
}
printf("Addition of matrices is:\n");
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
printf("%d\t",C[i][j]);
}
printf("\n");
}
}
void subtraction(int m, int n, int A[m][n],int B[m][n])
{
int C[m][n];
int i,j;
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
C[i][j]=A[i][j]-B[i][j];
}
}
printf("Subtraction of matrices is:\n");
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
printf("%d\t",C[i][j]);
}
printf("\n");
}
}
void multiplication(int m, int n, int A[m][n],int B[m][n])
{
```

```
int C[m][n];
int i,j,k;
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
C[i][j]=0;
for(k=0;k<n;k++)
{
C[i][j]=C[i][j]+A[i][k]*B[k][j];
}
}
}
printf("Multiplication of matrices is:\n");
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
printf("%d\t",C[i][j]);
}
printf("\n");
}
}
void transpose(int m,int n,int A[m][n])
{
int i,j;
printf("Transpose of matrix A is:\n");
for(i=0;i<n;i++)
{
for(j=0;j<m;j++)
{
printf("%d\t",A[j][i]);
}
printf("\n");
}
}
void main()
{
int choice;
int m,n,i,j;
printf("Enter number of rows in matrix A,B\n");
scanf("%d",&m);
printf("Enter number of columns in matrix A,B\n");
scanf("%d",&n);
int A[m][n],B[m][n];
printf("Enter matrix A elements\n");
for(i=0;i<m;i++)
```

```
{
for(j=0;j<n;j++)
{
scanf("%d",&A[i][j]);
}
}
printf("Enter matrix B elements\n");
for(i=0;i<m;i++)
{
for(j=0;j<n;j++)
{
scanf("%d",&B[i][j]);
}
}
do
{
printf("Enter 1.Addition\n2.Subtraction\n3.Multiplication\n4.Transpose\n5.Exit\n");
printf("Enter your choice: ");
scanf("%d",&choice);
switch(choice)
{
case 1:
addition(m,n,A,B);
break;
case 2:
subtraction(m,n,A,B);
break;
case 3:
multiplication(m,n,A,B);
break;
case 4:
transpose(m,n,A);
break;
case 5:
printf("Exiting the code\n");
break;
default:
printf("Wrong choice is taken\n");
break;
}
}while(choice!=5);
}
```

Assinment 2 sparse matrix

```
#include <stdio.h>
void display_sparse(int m, int n, int A[m][n])
{
   int i,j;
   printf("Original Sparse Matrix is:\n");
   for(i=0;i<m;i++)
   {
      for(j=0;j<n;j++)
      {
         printf("%d\t",A[i][j]);
      }
      printf("\n");
   }
}
void compact(int m, int n, int A[m][n], int B[m*n+1][3])
{
   int i,j,k=1;
  B[0][0]=m;
   B[0][1]=n;

   for(i=0;i<m;i++)
   {
      for(j=0;j<n;j++)
      {
         if(A[i][j]!=0)
         {
            B[k][0]=i;
            B[k][1]=j;
            B[k][2]=A[i][j];
            k++;
         }
      }
   }
   B[0][2]=k-1;
}

void display(int rows, int B[rows][3])
{
   int i,j;
   for(i=0;i<=B[0][2];i++)
   {
      for(j=0;j<3;j++)
      {
         printf("%d\t",B[i][j]);
      }
      printf("\n");
   }
}
```

```
void simple_transpose(int rows, int B[rows][3])
{
   int C[rows][3];
   int i,j,k=1;
   C[0][0]=B[0][1];
   C[0][1]=B[0][0];
   C[0][2]=B[0][2];
   for(i=0;i<B[0][1];i++)
   {
      for(j=1;j<=B[0][2];j++)
      {
         if(B[j][1]==i)
         {
            C[k][0]=B[j][1];
            C[k][1]=B[j][0];
            C[k][2]=B[j][2];
            k++;
         }
      }
   }
   printf("Simple Transpose is:\n");
   display(rows,C);
}
void fast_transpose(int rows, int n, int B[rows][3])
{
   int C[rows][3];
   int s[n],t[n];
   int i,j,k;

   for(i=0;i<n;i++)
   {
      s[i]=0;
   }
   for(i=1;i<=B[0][2];i++)
   {
      s[B[i][1]]++;
   }
   t[0]=1;
   for(i=1;i<n;i++)
   {
      t[i]=t[i-1]+s[i-1];
   }
   C[0][0]=B[0][1];
   C[0][1]=B[0][0];
   C[0][2]=B[0][2];
   for(i=1;i<=B[0][2];i++)
   {
      j=B[i][1];
      k=t[j];
      C[k][0]=B[i][1];
      C[k][1]=B[i][0];
```

```
        C[k][2]=B[i][2];
        t[j]++;
    }
    printf("Fast Transpose is:\n");
    display(rows,C);
}

int main()
{
    int m,n,i,j,choice;
    printf("Enter number of rows:\n");
    scanf("%d",&m);
    printf("Enter number of columns:\n");
    scanf("%d",&n);
    if(m<=0 || n<=0)
    {
        printf("Invalid matrix size\n");
        return 1;
    }
    int A[m][n];
    int B[m*n+1][3];
    printf("Enter matrix elements:\n");
    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)
        {
            scanf("%d",&A[i][j]);
        }
    }
    // Display original sparse matrix automatically
    display_sparse(m,n,A);
    compact(m,n,A,B);
    do
    {
        printf("\nEnter 1.Compact Form\n2.Simple Transpose\n3.Fast Transpose\n4.Exit\n");
        printf("Enter your choice: ");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
                printf("Compact Form is:\n");
                display(m*n+1,B);
                break;

            case 2:
                simple_transpose(m*n+1,B);
                break;

            case 3:
                fast_transpose(m*n+1,n,B);
                break;
```

```
        case 4:
            printf("Exiting the code\n");
            break;

        default:
            printf("Wrong choice is taken\n");
            break;
      }
   }while(choice!=4);
   return 0;
}
```

Assignment 3

```
#include <stdio.h>
struct Student
{
   int roll;
   char name[50];
   float marks;
};
struct Student s[100];
int n;
void inputStudents()
{
   int i;
   printf("Enter number of students: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
   {
      printf("\nStudent %d\n", i + 1);
      printf("Enter Roll Number: ");
      scanf("%d", &s[i].roll);

      printf("Enter only first Name: ");
      scanf("%49s", s[i].name);

      printf("Enter Marks: ");
      scanf("%f", &s[i].marks);
   }
}

void Students()
{
   int i;
   printf("Roll\tName\tMarks\n");

   for(i = 0; i < n; i++)
   {
```

```
        printf("%d\t%s\t%.2f\n", s[i].roll, s[i].name, s[i].marks);
    }
}

void linearSearch()
{
    int roll, i, found = 0;

    printf("Enter Roll Number to search: ");
    scanf("%d", &roll);

    for(i = 0; i < n; i++)
    {
        if(s[i].roll == roll)
        {
            printf("\nStudent Found\n");
            printf("Roll: %d\nName: %s\nMarks: %.2f\n",
                  s[i].roll, s[i].name, s[i].marks);
            found = 1;
            break;
        }
    }

    if(!found)
        printf("Student not found.\n");
}

void insertionSort()
{
    int i, j;
    struct Student key;

    for(i = 1; i < n; i++)
    {
        key = s[i];
        j = i - 1;

        while(j >= 0 && s[j].roll > key.roll)
        {
            s[j + 1] = s[j];
            j = j - 1;
        }

        s[j + 1] = key;
    }

    printf("Records sorted using Insertion Sort on roll number.\n");
    Students();
}

void selectionSort()
```

```
{
   int i, j, min;
   struct Student temp;

   for(i = 0; i < n - 1; i++)
   {
      min = i;

      for(j = i + 1; j < n; j++)
      {
         if(s[j].roll < s[min].roll)
            min = j;
      }

      if(min != i)
      {
         temp = s[i];
         s[i] = s[min];
         s[min] = temp;
      }
   }

   printf("Records sorted using Selection Sort on roll number.\n");
   Students();
}

void shellSort()
{
   int gap, i, j;
   struct Student temp;

   for(gap = n / 2; gap > 0; gap = gap / 2)
   {
      for(i = gap; i < n; i++)
      {
         temp = s[i];

         for(j = i; j >= gap && s[j - gap].roll > temp.roll; j = j - gap)
         {
            s[j] = s[j - gap];
         }

         s[j] = temp;
      }
   }

   printf("Records sorted using Shell Sort on roll number.\n");
   Students();
}

void binarySearch()
```

```
{
   int roll;
   int low = 0, high = n - 1, mid;

   printf("Enter Roll Number to search: ");
   scanf("%d", &roll);

   while(low <= high)
   {
      mid = (low + high) / 2;

      if(s[mid].roll == roll)
      {
         printf("\nStudent Found\n");
         printf("Roll: %d\nName: %s\nMarks: %.2f\n",
              s[mid].roll, s[mid].name, s[mid].marks);
         return;
      }
      else if(roll < s[mid].roll)
      {
         high = mid - 1;
      }
      else
      {
         low = mid + 1;
      }
   }

   printf("Student not found.\n");
}

int main()
{
   int choice;

   inputStudents();

   do
   {
      printf("\n1. Display Students\n");
      printf("2. Linear Search\n");
      printf("3. Insertion Sort\n");
      printf("4. Selection Sort\n");
      printf("5. Shell Sort\n");
      printf("6. Binary Search\n");
      printf("7. Exit\n");

      printf("Enter your choice: \n");
      scanf("%d", &choice);

      switch(choice)
```

```
    {
        case 1:
            Students();
            break;

        case 2:
            linearSearch();
            break;

        case 3:
            insertionSort();
            break;

        case 4:
            selectionSort();
            break;

        case 5:
            shellSort();
            break;

        case 6:
            printf("Data should be sorted.\n");
            binarySearch();
            break;

        case 7:
            printf("End code.\n");
            break;

        default:
            printf("Invalid choice!\n");
    }

  } while(choice != 7);

  return 0;
}
```

Assignment 6

Choice	Input	Output
1. Infix to Postfix	`A+B*C`	`ABC*+`
2. Prefix to Infix	`+A*BC`	`(A+(B*C))`
3. Prefix to Postfix	`+A*BC`	`ABC*+`
4. Postfix to Infix	`ABC*+`	`(A+(B*C))`

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

struct Stack
{
   char data[MAX][MAX];
   int top;
};

void init(struct Stack *s)
{
   s->top = -1;
}

int isEmpty(struct Stack *s)
{
   return s->top == -1;
}

void push(struct Stack *s, char item[])
{
   if(s->top == MAX - 1)
   {
      printf("Stack Overflow\n");
      return;
   }

   s->top++;
   strcpy(s->data[s->top], item);
}

void pop(struct Stack *s, char item[])
{
   if(isEmpty(s))
   {
      printf("Stack Underflow\n");
      return;
   }

   strcpy(item, s->data[s->top]);
   s->top--;
}

int precedence(char op)
{
   if(op == '^')
```

```
      return 3;

   if(op == '*' || op == '/')
      return 2;

   if(op == '+' || op == '-')
      return 1;

   return 0;
}

void reverse(char str[])
{
   int i, j;
   char temp;

   i = 0;
   j = strlen(str) - 1;

   while(i < j)
   {
      temp = str[i];
      str[i] = str[j];
      str[j] = temp;
      i++;
      j--;
   }
}

void infixToPostfix(char infix[], char postfix[])
{
   struct Stack s;
   int i, k = 0;
   char item[2];

   init(&s);

   for(i = 0; infix[i] != '\0'; i++)
   {
      if(isalnum(infix[i]))
      {
         postfix[k++] = infix[i];
      }
      else if(infix[i] == '(')
      {
         item[0] = infix[i];
         item[1] = '\0';
         push(&s, item);
      }
      else if(infix[i] == ')')
      {
```

```
            while(!isEmpty(&s) && s.data[s.top][0] != '(')
            {
               pop(&s, item);
               postfix[k++] = item[0];
            }

            if(!isEmpty(&s))
            {
               pop(&s, item);
            }
         }
         else
         {
            while(!isEmpty(&s) &&
                  s.data[s.top][0] != '(' &&
                  precedence(s.data[s.top][0]) >= precedence(infix[i]))
            {
               pop(&s, item);
               postfix[k++] = item[0];
            }

            item[0] = infix[i];
            item[1] = '\0';
            push(&s, item);
         }
      }

      while(!isEmpty(&s))
      {
         pop(&s, item);
         postfix[k++] = item[0];
      }

      postfix[k] = '\0';
   }

   void prefixToInfix(char prefix[], char result[])
   {
      struct Stack s;
      int i;
      char op1[MAX], op2[MAX], temp[MAX];
      char item[2];

      init(&s);

      for(i = strlen(prefix) - 1; i >= 0; i--)
      {
         if(isalnum(prefix[i]))
         {
            item[0] = prefix[i];
            item[1] = '\0';
```

```
            push(&s, item);
        }
        else
        {
            pop(&s, op1);
            pop(&s, op2);

            sprintf(temp, "(%s%c%s)", op1, prefix[i], op2);
            push(&s, temp);
        }
    }

    pop(&s, result);
}

void prefixToPostfix(char prefix[], char result[])
{
    struct Stack s;
    int i;
    char op1[MAX], op2[MAX], temp[MAX];
    char item[2];

    init(&s);

    for(i = strlen(prefix) - 1; i >= 0; i--)
    {
        if(isalnum(prefix[i]))
        {
            item[0] = prefix[i];
            item[1] = '\0';
            push(&s, item);
        }
        else
        {
            pop(&s, op1);
            pop(&s, op2);

            sprintf(temp, "%s%s%c", op1, op2, prefix[i]);
            push(&s, temp);
        }
    }

    pop(&s, result);
}

void postfixToInfix(char postfix[], char result[])
{
    struct Stack s;
    int i;
    char op1[MAX], op2[MAX], temp[MAX];
    char item[2];
```

```
    init(&s);

    for(i = 0; postfix[i] != '\0'; i++)
    {
        if(isalnum(postfix[i]))
        {
            item[0] = postfix[i];
            item[1] = '\0';
            push(&s, item);
        }
        else
        {
            pop(&s, op2);
            pop(&s, op1);

            sprintf(temp, "(%s%c%s)", op1, postfix[i], op2);
            push(&s, temp);
        }
    }

    pop(&s, result);
}

int main()
{
    int choice;
    char expression[MAX];
    char result[MAX];

    while(1)
    {
        printf("\nExpression Conversion\n");
        printf("1. Infix to Postfix\n");
        printf("2. Prefix to Infix\n");
        printf("3. Prefix to Postfix\n");
        printf("4. Postfix to Infix\n");
        printf("5. Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter infix expression: ");
                scanf("%99s", expression);

                infixToPostfix(expression, result);
                printf("Postfix = %s\n", result);
                break;
```

```
            case 2:
                printf("Enter prefix expression: ");
                scanf("%99s", expression);

                prefixToInfix(expression, result);
                printf("Infix = %s\n", result);
                break;

            case 3:
                printf("Enter prefix expression: ");
                scanf("%99s", expression);

                prefixToPostfix(expression, result);
                printf("Postfix = %s\n", result);
                break;

            case 4:
                printf("Enter postfix expression: ");
                scanf("%99s", expression);

                postfixToInfix(expression, result);
                printf("Infix = %s\n", result);
                break;

            case 5:
                printf("Exiting the code\n");
                exit(0);

            default:
                printf("Invalid choice!\n");
        }
    }

    return 0;
}
```

Assignment 7 Postfix conversion

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX 100

struct Stack
{
    int data[MAX];
    int top;
};
```

```
void init(struct Stack *s)
{
   s->top = -1;
}

int isEmpty(struct Stack *s)
{
   return s->top == -1;
}

void push(struct Stack *s, int value)
{
   if(s->top == MAX - 1)
   {
      printf("Stack Overflow\n");
      return;
   }

   s->top++;
   s->data[s->top] = value;
}

int pop(struct Stack *s)
{
   if(isEmpty(s))
   {
      printf("Stack Underflow\n");
      return -1;
   }

   return s->data[s->top--];
}

int calculate(int a, int b, char op)
{
   switch(op)
   {
      case '+':
         return a + b;

      case '-':
         return a - b;

      case '*':
         return a * b;

      case '/':
         return a / b;

      case '%':
         return a % b;
```

```
        default:
            printf("Invalid operator!\n");
            return 0;
    }
}

int evaluatePostfix(char expression[])
{
    struct Stack s;
    int i;
    int operand1, operand2, result;

    init(&s);

    for(i = 0; expression[i] != '\0'; i++)
    {
        if(isdigit(expression[i]))
        {
            push(&s, expression[i] - '0');
        }
        else
        {
            operand2 = pop(&s);
            operand1 = pop(&s);

            result = calculate(operand1, operand2, expression[i]);

            push(&s, result);
        }
    }

    return pop(&s);
}

int main()
{
    char expression[MAX];
    int result;

    printf("Enter postfix expression: ");
    scanf("%99s", expression);

    result = evaluatePostfix(expression);

    printf("Result = %d\n", result);

    return 0;
}
```